

UF01 - Acceso a ficheros

1. Tipos de ficheros	2
Ficheros de texto	2
Ficheros de texto plano	3
Ficheros de configuración	3
Ficheros XML	3
2. Acceso a ficheros	4
Apertura y cierre de ficheros con try-with-resources.....	4
Lectura de ficheros de texto plano	5
Escritura de ficheros de texto plano	7
Manipulación de fichero binarios en Java	8
Manipulación de archivos CSV	12
3. Directorios	17
La clase File	17
Rutas y dependencias del sistema operativo.....	17
4. Tratamiento de archivos XML en Java.....	18
Lectura sobre XML	20
Movernos por el árbol DOM	20
Modificar un texto y un atributo en un XML.....	22
Eliminar contenido en un XML	22
Crear Elementos/Sub-elementos en un XML.....	23
Escritura en archivo XML	24

1. Tipos de ficheros

Desde el punto de vista de un programador solamente distinguiremos entre dos tipos de ficheros:

- **Ficheros de texto** cuando el contenido del fichero contenga exclusivamente caracteres de texto (podemos leerlo con un simple editor de texto)

Extensión	Tipo de fichero
.txt	Fichero de texto plano
.xml	Fichero XML
.json	Fichero de intercambio de información
.props	Fichero de propiedades
.conf	Fichero de configuración
.sql	Script SQL
.srt	Fichero de subtítulo

- **Ficheros binarios** cuando no estén compuestos exclusivamente de texto. Pueden contener imágenes, vídeos, ficheros, . . . aunque también podemos considerar un fichero binario a un fichero de Microsoft Word en el que sólo hayamos escrito algún texto puesto que, al almacenarse el fichero, el procesador de texto incluye alguna información binaria.

Ficheros de texto

En esta parte vamos a trabajar con 3 tipos de ficheros de texto:

- **Ficheros de texto plano** que contendrán texto *libre* y donde podremos escribir sin respetar ningún tipo de formato
- **Ficheros de configuración** que contendrán información de configuración para una aplicación. Tienen un formato específico y Java además proporciona una API para trabajar más cómodamente con ellos

- **Ficheros XML** que contienen información y están acompañados de etiquetas que le dan significado. Tienen unas reglas y formato más o menos definido y Java proporciona una API para trabajar con ellos

Existen más tipos de ficheros de texto también muy extendidos como *.json* y *.csv* pero no serán estudiados en esta parte.

Ficheros de texto plano

Los ficheros de texto son aquellos que únicamente contienen texto, por lo que pueden ser editados directamente con cualquier editor de texto plano (Bloc de Notas, Notepad++, . . .). Se podría decir que son aquellos ficheros que podrían ser *leídos* por cualquier persona. Son aquellos que normalmente se almacenan con la extensión *.txt* pero también podríamos incluir los scripts SQL (*.sql*), ficheros de código Java (*.java*), ficheros de configuración (*.ini*, *.props*, *.conf*,...).

También se incluyen en la categoría de ficheros de texto los que además incluyen información adicional (siempre en forma de texto) que permiten interpretar los datos del fichero de una manera u otra, añadiendo más información al mismo. Estos formatos son HTML, XML, JSON...

En cualquier caso, desde Java siempre se podrán leer/escribir de la misma manera, según veremos a continuación. También veremos cómo pueden leerse/escribirse utilizando librerías aquellos ficheros de texto plano que contienen formato, como hemos comentado anteriormente, haciendo de esa forma mucho más fácil el trabajo.

Ficheros de configuración

En la API de Java se incluyen librerías para trabajar con los ficheros de configuración. Puesto que todos siguen un mismo patrón, es la librería la que se encarga de acceder al fichero a bajo nivel y el programador sólo tiene que indicar a qué propiedad quiere acceder o que propiedad quiere modificar, sin tener que añadir nada de código para leer o escribir el fichero tal y como hemos visto en el punto anterior.

Ficheros XML

Los ficheros XML permiten el intercambio de información entre aplicaciones utilizando para ello un fichero de texto plano al que se le pueden añadir etiquetas para darle significado a cada uno de los valores que se almacenan.

Por ejemplo, el siguiente fichero XML podría ser el resultado de volcar una Base de Datos sobre productos de una compañía, de forma que dicha información podría ahora leerse desde otra aplicación e incorporarla. Se puede ver como a parte de los datos de dichos productos (sus nombres, precios, . . .) aparece otra información en forma de etiquetas (entre los caracteres < y >) que permite dar significado a cada dato almacenado en el fichero. Así, podemos saber de qué estamos hablando y a qué corresponde cada valor.

Obviamente, tal y como ocurría con los ficheros de configuración, toda la información se tiene que pasar a texto para poder crear el fichero y reconvertida a su tipo original cuando se cargue de nuevo. Las mismas consideraciones que hemos tenido en cuenta antes nos servirán ahora.

2. Acceso a ficheros

La persistencia consiste precisamente en guardar los datos de forma que continúen disponibles después de finalizar el programa. Java proporciona diferentes mecanismos para trabajar con ficheros:

- Ficheros de texto.
- Ficheros CSV.
- Ficheros binarios.
- Flujos de entrada y salida.
- **Serialización** de objetos.

En Java moderno utilizaremos principalmente las clases del paquete:

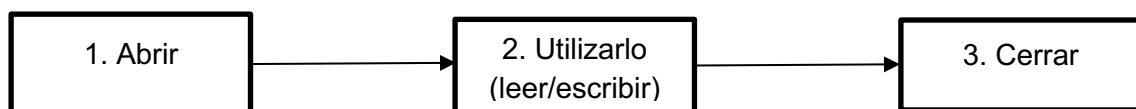
`java.nio.file`

aunque también veremos algunas clases tradicionales de:

`java.io`

Apertura y cierre de ficheros con try-with-resources

Cuando un programa necesita trabajar con un fichero, normalmente debe realizar tres operaciones:



Java proporciona diferentes clases para trabajar con ficheros. Por ejemplo, para leer texto podemos utilizar `FileReader`:

```
FileReader lector = new FileReader("datos.txt");
```

¿Por qué hay que cerrar el fichero al terminar de usarlo?

Cerrar un fichero es importante porque cuando lo abrimos Java obtiene recursos del sistema operativo, no estamos trabajando simplemente con una variable de Java.

Si un programa abre muchos ficheros y no los cierra, puede terminar consumiendo demasiados recursos del sistema.

Esto puede provocar problemas como:

- Consumo innecesario de recursos.

- Errores al intentar abrir más ficheros.
- Ficheros que permanecen bloqueados.
- Datos que todavía no se han terminado de escribir.
- Problemas especialmente graves en aplicaciones que trabajan continuamente con ficheros.

Por eso existe una regla fundamental: ***Todo recurso que se abre debe cerrarse cuando ya no se necesita.*** Para cerrar una variable que contiene una referencia al fichero utilizaremos la función `close()` del objeto.

```
lector.close();
```

Puesto que el cierre de un fichero es a veces difícil de gestionar, Java tiene un mecanismo que se encarga de cerrar los ficheros automáticamente al acabar de ejecutar un bloque de código concreto o si se produce un error al acceder al fichero o cuando acabamos de ejecutar el código que trata con él. Este se conoce como un bloque llamado `try-with-resources`.

```
try (
    FileReader lector = new FileReader("datos.txt");
){

    //Instrucciones de acceso al fichero

} catch (IOException e){
    e.printStackTrace();
}
```

Esta instrucción Java nos permite crear un objeto con el descriptor de fichero antes de comenzar un bloque `try` y nos asegura que este descriptor se cerrará automáticamente al finalizar el `try` o si se produce un error y el flujo de ejecución cambia.

IMPORTANTE: Dentro del bloque de los paréntesis del `try-with-resources` solo podrás crear objetos que sean del tipo **Closeable** y que estos solo podrán utilizarse dentro del bloque `try`.

Lectura de ficheros de texto plano

Java proporciona diferentes clases para trabajar con ficheros de texto. Dos de las más utilizadas tradicionalmente son:

- **FileReader:** permite abrir un fichero de texto y leer sus caracteres.
- **BufferedReader:** permite leer texto de forma más eficiente y, especialmente,

facilita la lectura línea a línea.

La clase **FileReader** (**java.io.FileReader**) tiene como función principal abrir un fichero de texto para poder leer su contenido carácter a carácter mediante la función `read()`. Esta función devuelve un `int` que contiene el código ASCII del carácter leído y `-1` cuando lee el final de fichero.

```
FileReader lector = new FileReader("datos.txt");

int caracter;

while ((caracter = lector.read()) != -1) {
    System.out.print((char) caracter);
}
```

Por otro lado tenemos la clase `BufferedReader` (`java.io.BufferedReader`) cuya función es proporcionar una lectura de texto más eficiente mediante un búfer. Con esta clase podemos leer cómodamente cada línea de un fichero utilizando la función `readLine()`. Esta función devuelve un `String` con la línea leída o `null` cuando lee el final del fichero.

Uniendo ambas clases tendremos podremos escribir un código como el siguiente:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerFichero {
    public static void main(String[] args) {
        try (
            FileReader fr = new FileReader("alumnos.txt");
            BufferedReader lector = new BufferedReader(fr);
        ) {
            String linea;

            while ((linea = lector.readLine()) != null) {
                System.out.println(linea);
            }

        } catch (IOException e) {
            System.out.println("Error al leer el fichero.");
        }
    }
}
```

Que lee cada línea de un fichero y la imprime por pantalla.

Escritura de ficheros de texto plano

Java proporciona diferentes clases para trabajar con ficheros de texto. Dos de las más utilizadas tradicionalmente para escribir información son:

- **FileWriter**: permite abrir un fichero de texto y escribir caracteres en él.
- **BufferedWriter**: permite escribir texto de forma más eficiente mediante un búfer y proporciona funciones para escribir texto y saltos de línea.

La clase **FileWriter** (**java.io.FileWriter**) tiene como función principal abrir un fichero de texto para poder escribir caracteres mediante la función `write()`. En este caso, la creación del objeto `FileWriter` puede recibir un parámetro extra de tipo boolean al llamar al constructor que indica el modo de escritura en el que queremos abrir el fichero: borrando todo el contenido para sobrescribirlo con un nuevo valor o añadiendo contenido al final del fichero.

```
// Abrir el fichero sobrescribiendo el contenido
FileWriter escritor1 = new FileWriter("datos.txt");
FileWriter escritor2 = new FileWriter("datos.txt",false);

// Abrir el fichero añadiendo contenido al final (append)
FileWriter escritor3 = new FileWriter("datos.txt",true);
```

Si el fichero indicado no existe, `FileWriter` lo crea. Sin embargo, si el fichero ya existe, **por defecto se elimina su contenido y se escribe desde el principio del fichero.**

Por otro lado tenemos la clase `BufferedWriter` (**java.io.BufferedWriter**) cuya función es proporcionar una escritura de texto más eficiente mediante un búfer. Esta clase se utiliza habitualmente junto con `FileWriter` y utilizaremos principalmente 2 funciones:

- `write(String)`: Escribe la cadena que le damos como parámetro en el fichero.
- `newLine()`: Inserta un salto de línea en el fichero.

Un ejemplo práctico de programa Java que hace uso de este mecanismo sería el siguiente:

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class EscribirFichero {

    public static void main(String[] args) {
```

```
try (  
    FileWriter fw = new FileWriter("alumnos.txt", false);  
    BufferedWriter escritor = new BufferedWriter(fw);  
) {  
    escritor.write("Ana Mena");  
    escritor.newLine();  
  
    escritor.write("Carlos Alcaraz");  
    escritor.newLine();  
  
    escritor.write("María de la O");  
    escritor.newLine();  
  
} catch (IOException e) {  
    System.out.println("Error al escribir el fichero.");  
}  
}
```

Este programa crea el fichero `alumnos.txt` si no existe y escribe tres líneas en él. Si el fichero ya existía, **su contenido anterior será sobrescrito**. En caso de querer añadir al final del fichero cambiaríamos el false de la llamada al constructor de FileWriter por un true.

Manipulación de fichero binarios en Java

Hasta ahora hemos trabajado con ficheros de texto, en los que la información se almacena como caracteres y podemos utilizar clases como FileReader y BufferedReader para leerlos.

Sin embargo, existen muchos tipos de información que no se almacenan como texto. Por ejemplo, las imágenes, los archivos de audio, los vídeos o determinados datos utilizados por los programas se almacenan habitualmente en formato binario.

Un fichero binario almacena la información como una secuencia de bytes. Por este motivo, para trabajar con ellos utilizaremos clases diferentes a las utilizadas para los ficheros de texto.

Las clases más importantes que podemos utilizar son:

- **FileInputStream**: permite abrir un fichero binario para leer sus bytes.
- **FileOutputStream**: permite abrir un fichero binario para escribir bytes.
- **DataInputStream**: permite leer datos de tipos primitivos de Java desde un fichero binario.

- **DataOutputStream**: permite escribir datos de tipos primitivos de Java en un fichero binario.

FileInputStream

La clase **FileInputStream** (**java.io.FileInputStream**) permite abrir un fichero para realizar una lectura de sus bytes.

```
FileInputStream entrada = new FileInputStream("datos.bin");
```

Una vez abierto el fichero podemos utilizar la función `read()` para leer sus bytes. Esta función devuelve un `int` que contiene el byte leído y devuelve `-1` cuando se alcanza el final del fichero.

```
try (
    FileInputStream entrada = new FileInputStream("datos.bin");
){

    int dato;
    while ((dato = entrada.read()) != -1) {
        System.out.println(dato);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

En este caso estamos leyendo directamente los bytes almacenados en el fichero de 1 en 1.

FileOutputStream

La clase **FileOutputStream** (**java.io.FileOutputStream**) permite abrir un fichero binario para escribir bytes. Para escribir estos bytes utilizaremos la función `write(int)`, donde cada entero representa los bytes correspondientes al código ASCII de los caracteres que queremos escribir.

```
try (
    FileOutputStream escritor= new FileOutputStream("datos.bin");
){
    escritor.write(65); // Carácter 'A' en ASCII
    escritor.write(66); // Carácter 'B' en ASCII
}

} catch (IOException e) {
    e.printStackTrace();
}
```

Al igual que ocurría con `FileWriter`, `FileOutputStream` permite indicar si queremos sobrescribir el fichero o añadir información al final utilizando su constructor con un segundo parámetros de tipo boolean:

```
FileOutputStream escritor = new FileOutputStream("datos.bin",true);
```

DataOutputStream y DataInputStream

Aunque `FileOutputStream` permite trabajar directamente con bytes, normalmente resulta más cómodo trabajar con los tipos de datos que proporciona Java. Para ello podemos utilizar **`DataOutputStream`** (`java.io.DataOutputStream`). Esta clase permite escribir diferentes tipos de datos directamente en un flujo binario.

- `writeInt(int)` → escribe un int.
- `writeDouble(double)` → escribe un double.
- `writeBoolean(boolean)` → escribe un boolean.
- `writeChar(char)` → escribe un char.
- `writeUTF(String)` → escribe una cadena de texto (String).

Por ejemplo, el siguiente código

```
try (
    FileOutputStream fos = new FileOutputStream("datos.bin");
    DataOutputStream salida = new DataOutputStream(fos,false);
){
    salida.writeUTF("Ana");
    salida.writeInt(20);
    salida.writeDouble(1.75);

} catch (IOException e) {
    e.printStackTrace();
}
```

almacena en el fichero datos.bin tres datos:

- Nombre: Ana
- Edad: 20
- Altura: 1.75

Estos datos no se almacenan como texto legible, sino en formato binario.

Si podemos escribir datos primitivos en un fichero en formato binario, tiene sentido que estos datos se puedan recuperar posteriormente. Para ello utilizaremos **DataInputStream (java.io.DataInputStream)**. Esta clase permite leer los diferentes tipos de datos que se hayan almacenado en formato binario. Dispone de las funciones correspondientes de lectura como:

- `readInt()` → lee un int.
- `readDouble()` → lee un double.
- `readBoolean()` → lee un boolean.
- `readChar()` → lee un char.
- `readUTF()` → lee un String.

En el caso de la lectura, es muy importante conocer la forma en la que los datos fueron escritos, ya que estos deben ser leídos en el mismo orden y utilizando los mismos tipos. Por ejemplo, para leer los datos escritos en el código anterior deberíamos utilizar el siguiente código:

```
try (
    FileInputStream fis = new FileInputStream("datos.bin");
    DataInputStream entrada = new DataInputStream(fis);
){
    String nombre = entrada.readUTF();
    int edad = entrada.readInt();
    double altura = entrada.readDouble();

} catch (IOException e) {
    e.printStackTrace();
}
```

En resumen, `FileInputStream` y `FileOutputStream` proporcionan el acceso básico al fichero mediante bytes, mientras que `DataInputStream` y `DataOutputStream` nos permiten trabajar directamente con los tipos de datos de Java.

Manipulación de archivos CSV

CSV significa **Comma-Separated Values** (valores separados por comas). Es un formato de texto utilizado habitualmente para almacenar información organizada en filas y columnas. Pese a que los valores suelen estar separados por el carácter ',', como indica su nombre, es habitual encontrarlos también separados por ';'.

Por ejemplo, un archivo alumnos.csv podría contener algo como:

```
nombre;edad;curso
Ana;20;DAW
Carlos;21;DAW
María;19;DAM
```

Cada línea representa un registro y los diferentes campos están separados por un carácter.

Puesto que el contenido de estos archivos es texto plano, la primera posibilidad para trabajar con CSV en Java es tratarlo como cualquier otro fichero de texto, utilizando las clases *FileReader*, *BufferedReader*, *FileWriter* y *BufferedWriter*. De esta forma, podríamos procesar cada línea con la función **split** de la clase **String**, obteniendo algo como esto:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerCSV {
    public static void main(String[] args) {
        try (
            FileReader fr = new FileReader("alumnos.csv");
            BufferedReader lector = new BufferedReader(fr);
        ) {
            String linea;
            while ((linea = lector.readLine()) != null) {

                String[] datos = linea.split(";");

                System.out.println("Nombre: " + datos[0]);
                System.out.println("Edad: " + datos[1]);
                System.out.println("Curso: " + datos[2]);

                System.out.println("-----");
            }
        } catch (IOException e) {
            System.out.println("Error al leer el fichero.");
        }
    }
}
```

Pese a que parece trivial tratar con este tipo de archivos, podemos encontrarnos algunas situaciones inesperadas como que el mismo carácter que actúa de separador está también como parte de un campo con múltiples palabras entrecomillado ("").

```
nombre;direccion;edad  
Ana;"Calle Mayor; 25";20
```

Esto nos puede suponer una barrera mayor de esfuerzo para procesar este tipo de situaciones. Es por esto, que en algunos casos es conveniente usar algunas librerías externas (ya que Java no tiene ninguna librería propia que lo haga) para tratar con archivos CSV. En este caso veremos la librería OpenCSV y como puede simplificar nuestro trabajo, aunque también disponemos de otras librerías dedicadas al manejo de este tipo de archivos como Apache Commons CSV.

Añadir OpenCSV al proyecto

OpenCSV es una librería de Java especializada en la lectura y escritura de archivos CSV. Su principal ventaja frente a tratar el CSV directamente con **BufferedReader** y **split()** es que **se encarga de las particularidades del formato CSV**, como campos entrecomillados, separadores dentro de un campo o registros que ocupan varias líneas como veíamos en el ejemplo anterior.

Antes de nada, puesto que OpenCSV es una librería externa, debemos añadirla a nuestro proyecto Java. Para ello, si nuestro proyecto hace uso de Maven para gestionar las dependencias añadiremos la siguiente dependencia al archivo **pom.xml**:

```
<dependency>  
  <groupId>com.opencsv</groupId>  
  <artifactId>opencsv</artifactId>  
  <version>5.12.0</version>  
</dependency>
```

En caso de no usar un gestor de dependencias, simplemente deberemos descargar la librería y añadirla a nuestra carpeta **lib** del proyecto. La última versión disponible actualmente es la 5.12.0 y puede descargarse desde la siguiente URL:

<https://sourceforge.net/projects/opencsv/files/opencsv/5.12.0>

Leer un CSV con CSVReader

La clase principal de OpenCSV para leer archivos CSV es **CSVReader**.

Por ejemplo, imaginemos un archivo `alumnos.csv` con el siguiente contenido:

```
nombre;edad;nota
Ana;20;8.5
Carlos;21;6.75
María;19;9.25
```

Podemos leerlo así con el siguiente código:

```
import com.opencsv.CSVReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerCSV {
    public static void main(String[] args) {
        try (
            FileReader fr = new FileReader("alumnos.csv");
            CSVReader lector = new CSVReader(fr, ';');
        ) {
            String[] datos;

            while ((datos = lector.readNext()) != null) {
                for (String dato : datos) {
                    System.out.print(dato + " | ");
                }
                System.out.println();
            }
        } catch (IOException e) {
            System.out.println("Error al leer el fichero.");
        }
    }
}
```

readNext() devuelve un array de Strings (**String[]**) que contiene los campos del registro leído, como si aplicáramos la función **split**. OpenCSV documenta **CSVReader** precisamente como un lector que convierte cada registro CSV en un array de Strings.

Por tanto, para la línea **"Ana;20;8.5"** obtendríamos conceptualmente:

`datos[0] → "Ana"`

`datos[1] → "20"`

`datos[2] → "8.5"`

Por defecto, OpenCSV utiliza la coma ' , ' como separador. Sin embargo, en muchos CSV utilizados en España es habitual encontrar ' ; ' . Es por esto, que el constructor de CSVReader admite un segundo parámetro que recibe el carácter separador cuando no queramos utilizar la coma, como vemos en el ejemplo.

Esto resulta especialmente útil para trabajar con archivos generados por Excel u otras aplicaciones que utilizan ' ; ' como separador.

Por ahora todo parece funcionar como la primera aproximación que hemos comentado pero, ¿qué ocurre cuando el separador aparece como parte de un campo entre comillas como veíamos anteriormente? Esta es una de las principales ventajas de utilizar una librería especializada. Podemos tener:

```
nombre;direccion;edad  
Ana;"Calle Mayor; 25";20
```

Como veíamos antes, el segundo campo contiene un ' ; ' , pero está entre comillas. Con split, tendríamos un problema porque obtendríamos 4 Strings:

- Ana
- "Calle Mayor
- 25"
- 20

OpenCSV, en cambio, interpreta correctamente las comillas y el separador sin que nosotros tengamos que hacer NADA. Además, también puede trabajar con campos que contienen saltos de línea. La propia implementación de **CSVReader** contempla registros CSV que ocupan varias líneas. Esta es precisamente una de las razones para utilizar OpenCSV en lugar de implementar nosotros mismos el análisis del CSV.

Escribir un CSV con CSVWriter

Para escribir CSV podemos utilizar la clase **CSVWriter**. Tras abrir el archivo mediante **CSVWriter**, podremos escribir registros utilizando la función **writeNext(String[])** que recibirá un array con los campos que queremos escribir (por defecto los escribirá separados con el carácter ' , '). El código que se muestra a continuación escribiría el CSV que veíamos en la sección anterior:

```
import com.opencsv.CSVWriter;  
import java.io.FileWriter;  
import java.io.IOException;  
  
public class EscribirCSV {  
    public static void main(String[] args) {
```

```

try (
    FileWriter fw = new FileWriter("alumnos.csv");
    CSVWriter escritor = new CSVWriter(fw)
) {
    escritor.writeNext(new String[]{"nombre", "edad", "nota"});
    escritor.writeNext(new String[]{"Ana", "20", "8.5"});
    escritor.writeNext(new String[]{"Carlos", "21", "6.75"});
    escritor.writeNext(new String[]{"María", "19", "9.25"});

} catch (IOException e) {
    System.out.println("Error al escribir el fichero.");
}
}
}

```

CSVWriter también permite configurar separador, carácter de comillas, carácter de escape y final de línea utilizando su constructor completo como se puede ver en este fragmento de código:

```

CSVWriter escritor = new CSVWriter(
    new FileWriter("alumnos.csv"),           // Writer que indica el archivo
    ';',                                     // Carácter separador
    CSVWriter.DEFAULT_QUOTE_CHARACTER,       // Carácter de comillas
    CSVWriter.DEFAULT_ESCAPE_CHARACTER,     // Carácter de escape
    CSVWriter.DEFAULT_LINE_END              // String de final de línea
);

```

BufferedReader/BufferedWriter vs OpenCSV

Buffered	OpenCSV
Trabaja con texto	Trabaja con CSV
Devuelve líneas (String)	Devuelve registros (String[])
Nosotros debemos separar los campos	OpenCSV separa los campos
Podemos/necesitamos utilizar <code>split()</code>	No necesitamos <code>split()</code>
No conoce las reglas CSV	Conoce las reglas CSV
Forma parte de Java	Librería externa

3. Directorios

La clase File

Hasta ahora hemos utilizado clases como `FileReader`, `BufferedReader`, `FileWriter` y `BufferedWriter` para leer y escribir el contenido de los ficheros.

Java también proporciona la clase **File** (`java.io.File`), que permite trabajar con ficheros y directorios. Esta clase no se utiliza principalmente para leer o escribir el contenido de un fichero, sino para **representar una ruta del sistema de archivos** y consultar o modificar determinadas propiedades de ficheros y directorios. Un objeto `File` solo representa el archivo del nombre que indica, no comprueba si existe ni ningún tipo de información referente a este.

La clase `File` contiene una serie de utilidades para que nuestro programa pueda navegar por el sistema de archivos, así como para crear carpetas y ficheros. Las funciones más relevantes de esta clase son las siguientes:

Función	Descripción
<code>exists()</code>	Comprueba si existe
<code>isFile()</code>	Comprueba si es un fichero
<code>isDirectory()</code>	Comprueba si es un directorio
<code>getName()</code>	Obtiene el nombre
<code>getPath()</code>	Obtiene la ruta utilizada
<code>getAbsolutePath()</code>	Obtiene la ruta absoluta
<code>length()</code>	Obtiene el tamaño en bytes
<code>mkdir()</code>	Crea un directorio
<code>mkdirs()</code>	Crea un directorio y sus directorios padre
<code>delete()</code>	Elimina un fichero o directorio
<code>list()</code>	Obtiene los nombres (Strings) de los elementos de un directorio
<code>listFiles()</code>	Obtiene los elementos de un directorio como objetos File

Rutas y dependencias del sistema operativo

Cuando trabajamos con ficheros, existe un problema importante: **cada sistema operativo utiliza una forma diferente de representar las rutas.**

Por ejemplo, en Windows es habitual utilizar:

```
C:\Users\Alumno\Documentos\datos.txt
```

Mientras que en Linux y macOS se utilizan rutas como:

```
/home/alumno/Documentos/datos.txt
```

El separador utilizado en Windows es '\' mientras que en Linux y macOS es '/'. Por este motivo, no es recomendable escribir rutas en nuestro programa suponiendo que siempre vamos a utilizar un sistema operativo concreto, ya que esto puede provocar problemas:

Si queremos que nuestro programa sea independiente del sistema operativo, Java proporciona mecanismos para construir las rutas de forma adecuada: la constante **File.separator**.

Si queremos construir la ruta `./folder/file.txt` de manera independiente del sistema operativo podremos crearla de la siguiente manera:

```
String ruta = "." + File.separator + "folder" +  
File.separator + "file.txt";
```

Aunque `File.separator` es una solución válida y es importante conocerla porque forma parte de la clase `File`, Java proporciona actualmente una forma más recomendable de trabajar con rutas: las clases `Path` y `Paths` del paquete `java.nio.file`. Por ejemplo:

```
import java.nio.file.Path;  
import java.nio.file.Paths;  
  
Path ruta1 = Paths.get("datos", "alumnos", "alumnos.txt");  
Path ruta2 = Path.of("datos", "alumnos", "alumnos.txt");
```

generaría un objeto `Path` diferente dependiendo del sistema operativo en el que se ejecute el programa Java: en Windows la ruta será equivalente `datos\alumnos\alumnos.txt` y en Linux o macOS `datos/alumnos/alumnos.txt`. Ambos objetos (`ruta1` y `ruta2`) contendrán la misma ruta. En versiones modernas de Java, la función `Path.of()` suele ser la opción más sencilla.

4. Tratamiento de archivos XML en Java

XML (*eXtensible Markup Language*) es un formato de texto utilizado para almacenar e intercambiar información de forma estructurada. Los datos se organizan mediante elementos, atributos y una estructura jerárquica que permite representar relaciones entre ellos.

Java proporciona de forma integrada diferentes herramientas para trabajar con documentos XML. Para este tema utilizaremos principalmente **DOM** (*Document Object Model*), que permite cargar un documento XML en memoria y representarlo como un árbol de elementos. De esta forma, podemos leer, modificar, crear y eliminar elementos y atributos, y finalmente guardar los cambios en el fichero XML.

Para trabajar con DOM utilizaremos principalmente las siguientes clases incluidas en la API estándar de Java:

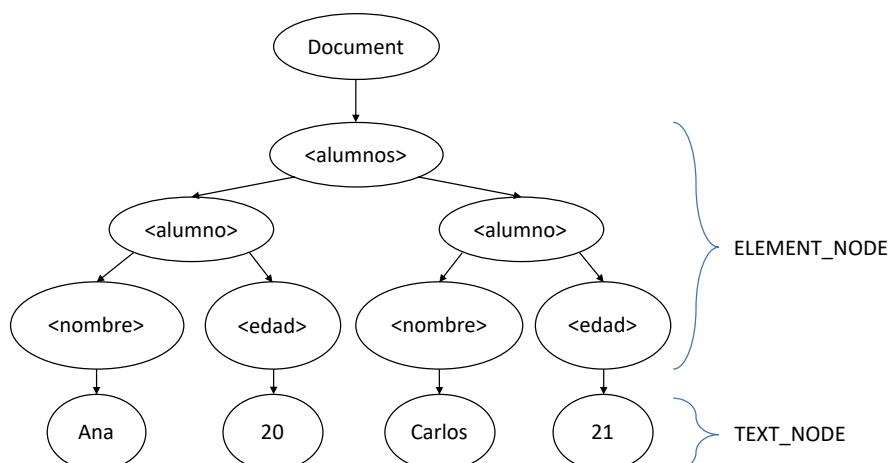
- **DocumentBuilderFactory:** Crear el parser XML.
- **DocumentBuilder:** Leer el XML y construir el árbol DOM.
- **Document:** Representar todo el documento XML en memoria.
- **Element:** Representar elementos XML.
- **Node:** Representar nodos del documento.
- **NodeList:** Trabajar con conjuntos de nodos.
- **TransformerFactory:** Crear el transformador.
- **Transformer:** Guardar el DOM modificado en un fichero.

Estas clases nos permitirán manipular documentos XML sin necesidad de instalar librerías externas.

Para abordar este apartado tenemos que conocer lo que hace DOM. DOM representa el XML como un árbol de objetos en memoria. Por ejemplo, para este XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos>
  <alumno id="1">
    <nombre>Ana</nombre>
    <edad>20</edad>
  </alumno>
  <alumno id="2">
    <nombre>Carlos</nombre>
    <edad>21</edad>
  </alumno>
</alumnos>
```

DOM nos permite imaginarlo así:



Lectura sobre XML

El primer paso sería cargar el documento:

```
import java.io.File;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.Document;

public class LeerXML {
    public static void main(String[] args) {
        try {
            // Crear los recursos necesarios para crear el parser
            DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
            DocumentBuilder builder = factory.newDocumentBuilder();

            // Crear la estructura de árbol
            Document documento = builder.parse(new File("alumnos.xml"));
            System.out.println("XML cargado correctamente");

        } catch (Exception e) {
            System.out.println("Error al leer el XML");
        }
    }
}
```

El paso clave de este código es la línea

```
Document documento = builder.parse(new File("alumnos.xml"));
```

Esta instrucción carga en memoria el XML y obtenemos un objeto `Document` con la estructura de árbol que podremos recorrer y modificar.

Movernos por el árbol DOM

Debemos tener en cuenta que un árbol DOM no es más que un conjunto de objetos tipo **Node** conectados unos a otros, cada uno con un id único asociado. Hay varias maneras de movernos por un árbol DOM para llegar a un lugar concreto dentro del objeto `Document` que genera el parser. Las funciones más comunes para hacerlo son las siguientes:

- **documento.getElementsByTagName("alumno")**: Devuelve una lista (**NodeList**) que contiene todos los **Node** con una etiqueta (Tag) concreta dentro del nodo en el que me encuentro. En este caso **documento** representa la raíz del árbol DOM y devuelve todos los nodos con la etiqueta **alumno** dentro de este. Solo puede usarse en objetos de tipo **Document** o **Element**.
- **node.getChildNodes()**: Devuelve un objeto **NodeList** con todos los nodos hijos del nodo **node**.

- **nodeList.getLength():** Devuelve la cantidad de nodos que contiene una lista de nodos **nodeList**. Es necesario para recorrer la lista.
- **nodeList.item(0):** Devuelve el nodo ubicado en la posición indicada en la llamada de la lista **nodeList**.
- **node.getNodeType():** Devuelve el tipo de nodo de **node**. Los tipos que nos interesará ver son principalmente **Node.ELEMENT_NODE** y **Node.TEXT_NODE**. Hay que tener en cuenta la existencia de la clase **Element**, que extiende la clase **Node** con una funcionalidad ampliada para nodos del tipo **ELEMENT_NODE**. De este modo, en algunas ocasiones, realizaremos una conversión de **Node** a **Element** para utilizar algunas funciones específicas.
- **node.getFirstChild()/node.getLastChild():** Devuelve el primer/último hijo del nodo **node**. Ésta es especialmente útil cuando sabemos que el nodo solo tiene un hijo y este es un **TEXT_NODE**.
- **node.getNodeName():** Devuelve un String con el nombre de la etiqueta del nodo **node** (el valor entre < y >). Si el nodo es un **TEXT_NODE** devuelve el texto **#text**.
- **node.getTextContent():** Devuelve el contenido textual que hay dentro del nodo, incluyendo el texto de sus nodos descendientes.
- **element.getAttribute("id"):** Esta función se utiliza en objetos del tipo **Element** y devuelve el valor del atributo con el nombre indicado como parámetro.

Es importante denotar que **NodeList** es un tipo de dato que no implementa **Iterable**, por lo que para recorrer una lista de nodos deberemos utilizar un bucle **for** con contador.

Un ejemplo en el que leemos del XML inicial la etiqueta y el valor de los nombres de los alumnos sería el siguiente:

```
public class LeerXML {
    public static void main(String[] args) {
        try {
            ...
            Document documento = builder.parse(new File("alumnos.xml"));
            NodeList alumnos = documento.getElementsByTagName("alumno");

            for (int i = 0; i < alumnos.getLength(); i++){
                Node alumno = alumnos.item(i);
                Node nombre = alumno.getFirstChild();
                System.out.println(nombre.getNodeName() + " -> " +
                                   nombre.getTextContent());
            }
        } catch (Exception e) {
            System.out.println("Error al leer el XML");
        }
    }
}
```

Modificar un texto y un atributo en un XML

Una vez somos capaces de encontrar el nodo que queremos modificar del XML como utilizando las funciones del apartado anterior, cambiar el valor del texto de un nodo es muy sencillo, basta con utilizar la función `node.setTextContent("nuevo valor")`.

También podemos modificar el valor de un atributo utilizando la función `element.setAttribute("id","10")`. NOTA: mientras la función `setTextContent` puede aplicarse a elementos de tipo `Node`, la función `setAttribute` solo puede aplicarse a tipos `Element`, por lo que deberemos hacer un casting de `Node` a `Element` para poder utilizarla.

```
public class LeerXML {
    public static void main(String[] args) {
        try {
            ...
            Document documento = builder.parse(new File("alumnos.xml"));

            NodeList alumnos = documento.getElementsByTagName("alumno");

            for (int i = 0; i < alumnos.getLength(); i++){
                Node alumno = alumnos.item(i);
                Node nombre = alumno.getFirstChild();

                if (nombre.getTextContent().equals("Ana")){
                    Element elem = (Element) alumno;
                    elem.setAttribute("id","10");
                    nombre.setTextContent("Ana Mena");
                }
            }
        } catch (Exception e) {
            System.out.println("Error al leer el XML");
        }
    }
}
```

Hay que tener en cuenta que esto cambia el nodo dentro del objeto documento, pero en ningún caso realizará la modificación en el archivo XML a no ser que nosotros realicemos una escritura como veremos más adelante.

Eliminar contenido en un XML

Hay varias cosas que podemos eliminar dentro de un XML:

- **Atributos de un nodo:** Si queremos eliminar un atributo de un nodo deberemos hacerlo sobre un objeto de la clase `Element`. La función que utilizaremos para hacerlo será `elem.removeAttribute("id")`. Ejemplo: Si tenemos el código XML `<alumno id="1">`, el siguiente código obtendría como resultado `<alumno>`.

```
...
Element alumno = alumnos.item(i);
alumno.removeAttribute("id");
...
```

- **Elementos/Sub-elementos de un elemento concreto:** Esto puede hacerse fácilmente haciendo uso de la función `element.removeChild(childNode)`. Esta función requiere primero encontrar el nodo que quieres eliminar (`childNode`) e indicarle al nodo padre (`node`) que lo elimine. Por ejemplo: en el bloque XML

```
<alumno id="1">
    <nombre>Ana</nombre>
    <edad>20</edad>
</alumno>
```

El código para eliminar el nodo edad sería:

```
...
Element alumno = alumnos.item(0);
NodeList edades = alumno.getElementsByTagName("edad");
Element edad = edades.item(0);
alumno.removeChild(edad);
...
```

Con una pequeña variante de este código podemos eliminar un elemento que tengamos localizado y todos sus sub-elementos. Para ello utilizaremos la función `getParentNode()`, que devuelve el nodo "padre" de otro nodo en el árbol. Si quisiéramos eliminar el nodo alumno entero bastaría con ejecutar la siguiente instrucción:

```
alumno.getParent().removeChild(alumno);
```

Crear Elementos/Sub-elementos en un XML

A la hora de crear elementos y sub-elementos es donde sucede algo muy interesante: la creación de los elementos no hace que pasen a formar parte del árbol, sino que debemos enlazarlos después de crearlos.

Imaginemos que queremos crear un nuevo bloque `<alumno>` y añadirlo a nuestro XML. Para ello, deberemos crear los elementos y sub-elementos de tipo `Element` (`Node.ELEMENT_NODE`) utilizando la función `document.createElement(String)`. Esta función necesitará hacer uso del objeto de tipo `Document` (`document`) donde está cargado nuestro XML.

De esta forma, podríamos crear un los nodos necesarios con el siguiente código:

```
Element alumno = document.createElement("alumno");
Element nombre = document.createElement("nombre");
Element edad = document.createElement("edad");
```

Este código crearía los elementos, pero ni crearía los valores, ni asociaría los nodos entre ellos, ni los incorporaría al XML. Veamos pues como completar el proceso:

1. Introducir los valores en los nodos nombre y edad. Para esto basta con utilizar la función **setTextContent** vista anteriormente:

```
nombre.setTextContent("Sergio");  
edad.setTextContent("38");
```

2. Enlazar los nodos nombre y edad al nodo alumno. Para esto utilizaremos la función **element.appendChild(childElem)** que enlaza un elemento como hijo de otro:

```
alumno.appendChild(nombre);  
alumno.appendChild(edad);
```

3. Añadir atributos si es necesario. Para ello utilizaremos la función **setAttribute** vista anteriormente.

```
alumno.setAttribute("id","3");
```

4. Enlazar el nodo alumno al XML. Para ello solo debemos encontrar un nodo que ya pertenezca al XML y volver a ejecutar **appendChild**. En este caso, el nuevo elemento **alumno** deberá colgar del elemento **alumnos**:

```
Element alumnos = documento.getElementsByTagName("alumnos").getFirstChild();  
alumnos.appendChild(alumno);
```

Esto completa el proceso de modificación del objeto que contiene el XML, ahora solo nos faltaría guardarlo en el archivo original o en un nuevo archivo .xml.

Escritura en archivo XML

Esta es la otra pieza fundamental del proceso. Después de modificar nuestro XML y hacer todos los cambios en la estructura del objeto **Document**, debemos escribirlo en un nuevo fichero.

Para ello utilizamos **TransformerFactory**, **Transformer**, **DOMSource** y **StreamResult**. Utilizaremos una serie de llamadas a estas clases similares a las que utilizamos para obtener la información del XML y convertirla a un objeto. Las llamadas serán las siguientes:

```
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;

public class GuardarXML{
    public void guardarXML(Document document){
        try {
            // Crear los recursos necesarios
            TransformerFactory factory = TransformerFactory.newInstance();
            Transformer transformer = factory.newTransformer();
            // Convertir el objeto Document a un formato "escribible"
            DOMSource source = new DOMSource(document);
            // Abrir y preparar el archivo de guardado
            // Si el archivo no existe el programa lo creará
            StreamResult result = new StreamResult("alumnos.xml");
            // Volcar el contenido del DOM al archivo
            transformer.transform(source, result);
        } catch (TransformerException e) {
            System.out.println("Error al guardar el documento XML.");
        }
    }
}
```